

AGENTS

INHERITED FROM Helix Constitution

Base agent rules live in the Helix Constitution submodule at the parent project's `constitution/AGENTS.md` and the universal `constitution/Constitution.md` it references. **READ THOSE FIRST.** The base file is authoritative for any topic not covered here. Module-specific rules below extend them; they never weaken them.

Critical universal rules every CLI agent (Claude Code, Cursor, Aider, Codex, Gemini CLI) MUST honour while working in this module:

- **No bluffing.** Every PASS carries positive evidence. Constitution §11.4.
- **Mutation-paired gates.** Every new gate has a paired mutation proving it catches regressions. Constitution §1.1.
- **No guessing language** (likely, probably, maybe, seems). Constitution §11.4.6.
- **Credentials never tracked.** `.env` patterns git-ignored; runtime-load only. Constitution §11.4.10.
- **Never force-push.** Force-push requires explicit per-session authorization AND a green §9.1.5 post-op gate. Constitution §9.
- **CONTINUATION.md kept in sync** in every non-trivial commit. Constitution §12.10.
- **60% RAM cap.** Heavy work wrapped in bounded execution scope. Constitution §12.6.

Canonical reference: <https://github.com/HelixDevelopment/Helix-Constitution>

AGENTS.md — ATMOSphere Lampa fork

This submodule inherits all parent ATMOSphere project AGENTS.md + Constitution invariants. The full canonical authority is parent docs/guides/ATMOSPHERE_CONSTITUTION.md. Read that first.

This file holds Lampa-specific agent guidance.

Mandatory build/commit/push constraints

1. **Build via parent** bash `scripts/build.sh --skip-pull --skip-tests --skip-ota` only. The parent's `step_build_lampa` is the canonical entrypoint; never invoke this fork's gradle directly outside that helper.
2. **Commit / push** via parent bash `scripts/commit_all.sh "message"` only. Never `git commit / git push` directly inside this submodule for source changes — the parent's deepest-first cascade catches the pointer change.
3. **Tags** cascade: every parent tag is mirrored on this submodule at HEAD via parent `scripts/testing/release_tag.sh`. Owned-submodule set is now 10 (was 9 — Lampa added 1.1.5-dev).

Lampa-specific guidance

- **applicationId stays top.rootu.lampa** — kept identical to upstream so existing user data / sources / preferences survive the rebrand. Renaming triggers PackageManager `uninstall+install` and wipes `/data/data/<pkg>/` (Fix #118 + Fix #124 same rationale).
- **Gradle 7.5.1 needs Java 17, not 21** — the build helper picks JDK 17 first. Java 21 (class file version 65) is rejected with "Unsupported class file major version 65".
- **Don't rebase** the fork against a much-newer upstream without diffing `AndroidManifest`, `build.gradle`, and `SigningConfigs` first — upstream may have switched to AGP 8.x / gradle 8.x which would reopen the JDK pick logic.
- **WebView playback path:** Lampa renders UI in a WebView and hands off media playback to external Android players (Kodi / VLC / MPV / Nova / TorrServe). Fix #88 / Fix #102 secondary-display routing applies via the chosen player's MediaC-odec — Lampa itself doesn't decode video. No subtitle forwarder needed inside Lampa.

- **Test coverage:** Lampa is fully covered by parent
pre_build_verification.sh Section CN-LAMPA gates +
meta_test_false_positive_proof.sh CM-LAMPA mutations +
post-flash test_lampa.sh.

MANDATORY ANTI-BLUFF COVENANT — END-USER QUALITY GUARANTEE (User mandate, 2026-04-28)

Forensic anchor — direct user mandate (verbatim):

"We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

This is the historical origin of the project's anti-bluff covenant. Every test, every Challenge, every gate, every mutation pair exists to make the failure mode (PASS on broken-for-end-user feature) mechanically impossible.

Operative rule: the bar for shipping is **not** "tests pass" but **"users can use the feature."** Every PASS in this codebase MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration- only PASS, "absence-of-error" PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Tests AND Challenges (HelixQA) are bound equally — a Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does. Both must produce positive end- user evidence; both are subject to the §8.1 five-constraint rule and §11 captured-evidence requirement.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §8.1 (positive-evidence-only validation) + §11 (bleeding-edge ultra-perfection quality bar) + §11.3 (the "no bluff" CLAUDE.md / AGENTS.md mandate) + **§11.4 (this end-user-quality-guarantee forensic anchor — propagation requirement enforced by pre-build gate CM-COVENANT-PROPAGATION).**

§11.4.1 extension (Phase 33, 2026-05-05) — FAIL-bluffs equally forbidden. A test that crashes for a script-internal reason (undefined variable under set -u, regex error, malformed assertion, missing argument) and produces a FAIL exit code is just as misleading as a PASS-bluff. Both let real defects ship

undetected. Per parent Constitution §11.4.1, every test MUST fail ONLY for genuine product defects — script-bug failures must be fixed at the source layer (helper library, shared lib, test source), not patched in individual call sites.

Non-compliance is a release blocker regardless of context.

§11.4.2 extension (Phase 34, 2026-05-06) — Recorded-evidence requirement. A test that emits PASS without captured visual or audio evidence of the user-visible feature actually working on the screen the user would see is a §11.4 PASS-bluff. Bug #13 (VK Video on PRIMARY display while a passing test claimed playback PASS) demonstrated the gap exactly. Closing it requires the recording + analyzer infrastructure (Bug #14 — `dual_display_record.sh / action_timeline.sh / Go recording-analyzer / helixqa-bridge`). Per Constitution §11.4.2 every PASS for a user-visible feature MUST be cross-checked by the analyzer against the dual-display recording + action timeline. A PASS that lacks at least one matched timeline event in the analyzer findings is treated as a §11.4 PASS-bluff.

Non-compliance is a release blocker regardless of context.

§11.4.3 extension (Phase 34, 2026-05-06) — Per-device-topology test dispatch. Tests that depend on hardware topology (secondary HDMI present/absent, microphone present/absent, etc.) MUST detect topology at test entry and dispatch the topology-appropriate variant. A test running the wrong variant for the actual topology and PASSing is a §11.4 PASS-bluff. Bug #18 (Lampa+TorrServe E2E) demonstrated the pattern: D1 (secondary HDMI) and D2 (primary only) get separate test variants behind a `dumpsys display`-based dispatcher. Per Constitution §11.4.3 every topology-touching test MUST have such a dispatcher OR explicit topology gates with SKIP-with-reason fallback.

Non-compliance is a release blocker regardless of context.

§11.4.4 extension (User mandate, 2026-05-06) — Test-interrupt-on-discovery + retest-from-clean-baseline. A test cycle that continues running past a freshly discovered defect is itself a §11.4 PASS-bluff: it produces "all green" summaries while the codebase under test is known-broken at the moment those greens were recorded. Phase 34.S' D1 demonstrated the violation when Bug #26 (hard-floor probe lifecycle) and Bug #27 (analyzer FAIL-bluff on non-video tests) were discovered mid-cycle and the cycle was allowed to continue, accumulating 13+ false-positive ANALYZER FAIL banners. Per Constitution §11.4.4 the moment any defect is re-discovered, re-produced, or newly identified during a test cycle, the cycle MUST stop on both devices. **Then:** (1) fix at root cause per §11.4.1, (2) land validation/verification tests for the fix — pre-build gate AND on-device test AND paired meta-test mutation, (3) full rebuild via `scripts/build.sh` (regardless of

whether the fix touched host script / Go binary / firmware — host-only fixes still get a full rebuild for retest baseline integrity), (4) re-flash D1 + D2, (5) repeat full `test_all_fixes.sh` from the beginning sequentially per §12.6, (6) end the cycle with `meta_test_false_positive_proof.sh` proving no gate is itself a bluff gate. Tests AND HelixQA Challenges are bound equally — Challenges that score PASS on a non-functional feature are the same class of defect as PASS-bluff unit tests; both must produce positive end-user evidence per §11.4.2 + §11.4.3.

Non-compliance is a release blocker regardless of context.

§11.4.4 expansion (User mandate, 2026-05-06) — Systematic debugging + four-layer test coverage + documentation + no-bluff certification. Augments the §11.4.4 base covenant with four non-negotiable additional requirements per the User mandate of 2026-05-06: (a) **Systematic debugging via superpowers skills.** Before applying any fix, run in-depth systematic debugging using the available `superpowers:*` skills (debugging, root-cause analysis, architectural-impact). Symptom patches are forbidden. The debugging output MUST identify root cause at source layer, blast radius across related tests/features/subsystems, and the regression-protection seam. (b) **Four-layer test coverage per fix.** Every fix lands with positive evidence in **every applicable layer**: pre-build gate (catches at source), post-build gate (catches in assembled image — proves bytes landed, cf. Fix #122 APK_LIB_MAP misroute), post-flash on-device test (fully automated, anti-bluff per §8.1, captured- evidence per §11.4.2, topology-dispatched per §11.4.3, orchestrator- wired in `test_all_fixes.sh`), HelixQA test bank entry (`banks/atmosphere.yaml` + per-feature additions), HelixQA full QA session coverage (Challenge-driven dispatch — bank entry without Challenge coverage is a §11.4 PASS-bluff), and meta-test paired mutation. Skipping a layer because "this fix only touches X" is forbidden. (c) **Documentation update for every fix.** Required: `docs/Issues.md` → `docs/Fixed.md` migration on closure, parent CLAUDE.md Applied Fixes Reference row, affected user-facing guides (`docs/guides/*.md`), affected diagrams/flowcharts/architecture docs, per-version `docs/changelogs/<tag>.md` entry. Documentation drift after a fix is itself a §11.4 violation. (d) **No-bluff certification per cycle.** Before tagging: `meta_test_false_positive_proof.sh` returns all gates green AND every gate's paired mutation FAILs (no bluff gates); `docs/Issues.md` open-set is empty or every entry explicitly classified out-of-scope-for-this-tag with operator sign-off (no known issues hidden); full suite returns zero new FAILs on either device (no working feature regressed); every gate has a paired mutation; every test produces positive evidence; every assertion catches its own negation (no error-prone or bluff-proof leftover).

Non-compliance is a release blocker regardless of context.

§11.4.5 — Audio + video quality analysis comprehensiveness (User mandate, 2026-05-07)

Forensic anchor — direct user mandate (verbatim, 2026-05-07):

"We MUST HAVE still analyzing of recorded materials and comprehensive validation and verification for issues we used to test! For example if there is audio at all or video, if so, is it good and proper or is it faulty? Does it have glitches, frame issues and other possible obstructions? IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected!"

§11.4.2 mandates *captured* evidence; §11.4.5 mandates the **content** of that evidence be analyzed for quality, not merely for presence. A test that captures a 0-byte mp4 (Bug #24) and PASSES because "the recording file exists" is the exact PASS-bluff pattern §11.4 forbids. Content-quality analysis is what closes that gap.

Audio quality analysis — every audio test that PASSES MUST verify ALL of: (1) **Presence** — non-trivial RMS amplitude in captured WAV / `/proc/asound/.../pcm*p/sub0/hw_params`. (2) **Channel count** — `ffprobe -show_streams` matches the test's claim (2.0 / 5.1 / 7.1). (3) **Sample rate + bit depth** — match the codec / pipeline under test. (4) **Glitch census** — XRUN / FastMixer under-run-overrun-partial / AudioFlinger `writeError` counts above tolerance MUST classify explicitly (PASS within budget, WARN above, FAIL on hard limits per §11.4.1 SKIP-vs-FAIL decision tree). (5) **Coexistence-artifact census** — for tests that exercise WiFi/BT alongside audio: BT TX queue overflow, A2DP src underflow, coex notification storms, 2.4 GHz radio contention.

Video quality analysis — every video test that PASSES MUST verify ALL of: (1) **Presence** — captured screen recording has non-zero file size AND `ffprobe -count_frames` reports decoded-frame total > 0. 0-byte mp4 (Bug #24) is the canonical PASS-bluff and triggers §11.4.4 STOP. (2) **Routing target** — analyzer + action-timeline confirms video appeared on the *intended* display (primary vs secondary HDMI; Bug #13 pattern). (3) **Frame health** — drop count, frame-time variance (jitter), freeze detection (SSIM > 0.99 for ≥ 1 s), tearing. (4) **Obstruction census** — Tesseract OCR scan for hostile overlays (Application not responding, Force close, sign-in dialog, geo-restriction overlay, ad break, paywall, App is not certified). (5) **Resolution + codec** — captured frame dimensions match the test's claim; downgrade is a PASS-bluff.

Challenges (HelixQA) are bound equally — every Challenge that asserts PASS MUST run all five audio + five video layers. A Challenge that scores PASS without applicable analysis is the same class of defect as a unit test that does.

Tooling guarantee: audio = tinycap + aplay --dump-hw-params + ffprobe + /proc/asound/parsers (lib/audio_validation.sh per §11.2.5). Video = screenrecord + ffprobe -count_frames + recording-analyzer + Tesseract OCR (scripts/dual_display_record.sh

- cmd/recording-analyzer/ per §11.4.2.A and §11.4.2.C). Tests dispatched against video evidence MUST honor §11.4.4 test-interrupt-on-discovery when the analyzer reports empty input — do not silently absorb that as a generic PASS-bluff banner.

Non-compliance is a release blocker regardless of context.

MANDATORY §12 HOST-SESSION SAFETY — INCIDENT #2 ANCHOR (2026-04-28)

Second forensic incident: on 2026-04-28 18:36:35 MSK the user's user@1000.service was again SIGKILLED (status=9/KILL), this time WITHOUT a kernel OOM kill (systemd-oomd inactive, MemoryMax=infinity) — a different vector than Incident #1. Cascade killed claude, tmux, the in-flight ATMOSPHERE build, and 20+ npm MCP server processes. Likely cumulative cgroup pressure + external watchdog.

Mandatory safeguards effective 2026-04-28 (full text in parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §12 Incident #2):

1. scripts/build.sh MUST source lib/host_session_safety.sh and call host_check_safety BEFORE any heavy step.
2. host_check_safety has 7 distress detectors including common cgroup-events warnings (#6) and current-boot session-kill events (#7).
3. Containers MUST be clean-slate destroyed + rebuilt after any suspected §12 incident. mem_limit is per-container, not per-user-slice — operator MUST cap $\sum \text{mem_limit} \leq \text{physical RAM} - \text{user-session overhead}$.
4. 20+ npm-spawned MCP server processes are a known memory multiplier; stop non-essential MCPs before heavy ATMOSPHERE work.
5. **Investigation: Docker/Podman as session-loss vector.** Per-container cgroups don't prevent cumulative user-slice pressure; common Failed to open cgroups file: /sys/fs/cgroup/

memory.events warnings preceded the 18:36:35 SIGKILL by 6 min — likely correlated.

This directive applies to every owned ATMOSphere repo and every HelixQA dependency. Non-compliance is a Constitution §12 violation.

MANDATORY §12.6 MEMORY-BUDGET CEILING — 60% MAXIMUM (User mandate, 2026-04-30)

Forensic anchor — direct user mandate (verbatim):

"We had to restart this session 3rd time in a row! The system of the host stays with no RAM memory for some reason! First make sure that whatever we do through our procedures related to this project **MUST NOT** use more than 60% of total system memory! All processes **MUST** be able to function normally!"

The mandate. Project procedures **MUST NOT** use more than **60% of total system RAM** (`HOST_SAFETY_MAX_MEM_PCT`). The remaining 40% is reserved for the operator's other workloads so the host can keep serving them while project work proceeds.

Three consecutive session-loss SIGKILLs on 2026-04-30 during 1.1.5-dev — every one happened while `scripts/build.sh` was running `m -j5 AOSP`. Each Soong/Ninja job peaks at ~5–8 GiB RSS; collective RSS overran the 60% envelope and the kernel OOM-killer escalated, taking down `user@1000.service`. **§12.1's pre-flight check (refusing to start if host already distressed) was not enough** — the missing piece was an active **CONSTRAINT** on heavy work itself.

Mandatory protections (rock-solid):

1. `HOST_SAFETY_MAX_MEM_PCT` defaults to 60 in `scripts/lib/host_session_safety.sh`.
2. `HOST_SAFETY_BUDGET_GB` is computed at source-time from `MemTotal × MAX_PCT/100`.
3. `bounded_run` clamps `MemoryMax` down to the budget if the caller asks for more (cgroup-level enforcement via `systemd-run --user --scope -p MemoryMax=...`).
4. `host_safe_parallel_jobs` and `host_safe_build_jobs` return the safe `-j` count given an estimated per-job RSS, capped at `nproc`.
5. `scripts/build.sh` wraps `m -j` in `bounded_run`. If the build's collective RSS exceeds the budget, only the scope is OOM-killed; `user@<uid>.service` stays alive.

Captured-evidence enforcement. Pre-build gate CM-MEMBUDGET-METATEST locks all 7 invariants and fires every pre-build run.

No escape hatch. §12.6 has NO operator-facing override flag. The cap exists for the operator's own protection; bypassing it is the bluff the §11.4 covenant specifically prohibits. Operators who need more headroom should reduce parallelism, close other workloads, or add RAM — NOT raise the percentage.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §12.6.

Non-compliance is a release blocker regardless of context.

MANDATORY §12.10 CONTINUATION DOCUMENT MAINTENANCE (User mandate, 2026-05-07)

Forensic anchor — direct user mandate (verbatim):

"during any work we perfrom, during Phases implementation, debugging and fixing, during ANY effort we have the Continuation document **MUST BE** maintained and it **MUST NOT BE** out of sync with current work we are doing! If for any reson we stop our work, we **MUST BE** able to continue any time, with current work, exactly where we have left of and from any CLI agent or any LLM model we chose! Nothing can be broken or faulty in maintained Continuation document!"

The mandate. A single, canonical, machine-readable handoff document — `docs/CONTINUATION.md` at the parent repo root — must always reflect the live state of the project. Any agent (human, Claude Code, Cursor, Aider, Codex, Gemini CLI, any future LLM) must be able to resume work **exactly where the previous session left off** by reading this single file.

Mandatory protections (no escape hatch):

1. `docs/CONTINUATION.md` **MUST exist** at the parent repo root.
2. **Every non-trivial state change** — work item started / completed / blocked, new bug discovered, phase transition, build state change, fix applied, gate added, mutation paired — **MUST update this document in the same commit** as the work itself. Commits that change source/tests/docs but leave `CONTINUATION.md` stale are non-compliant.
3. **Top-of-file Last updated:** ISO timestamp updated on every edit.
4. **Section §3 "Active work"** must list every IN PROGRESS / BLOCKED item with concrete commands, file paths, monitor

IDs, and percentages where relevant — enough that any agent can resume without conversation context.

5. **Section §0 "How to use this document"** must contain the verbatim resumption prompt — a single block any operator can paste into any CLI agent.
6. **Document MUST be self-contained** — no hyperlinks to ephemeral external systems as the only source of truth.

Captured-evidence enforcement. Pre-build gate CM-CONTINUATION-DOC-INSYNC locks 7 invariants (file exists, has Last-updated timestamp, timestamp recent enough, has §0 / §3 / §8 sections, line count ≥ 100). Pre-build gate CM-CONTINUATION-DOC-PROPAGATION verifies §12.10 text is present across parent CLAUDE.md / AGENTS.md and all 10 owned submodules' CLAUDE.md / AGENTS.md.

Commit-time enforcement. `scripts/commit_all.sh` refuses to commit if the staged change-set touches source / tests / gates but leaves `docs/CONTINUATION.md` untouched, with explicit override flag `--continuation-no-update-needed` (rationale captured in commit message).

Paired mutation. `meta_test_false_positive_proof.sh` CM-CONTINUATION-DOC-INSYNC mutation deletes the Last updated: line and asserts the gate FAILs.

No escape hatch. §12.10 has NO operator-facing override flag for the existence requirement. The discipline exists for the operator's own protection — the moment the document drifts from reality is the moment session-loss becomes catastrophic.

Canonical authority: parent `../../../../docs/guides/ATMOSPHERE_CONSTITUTION.md` §12.10.

Non-compliance is a release blocker regardless of context.

§11.4.6 — No-guessing mandate (User mandate, 2026-05-08)

Forensic anchor — direct user mandate (verbatim, 2026-05-08T18:30 MSK):

"'LIKELY' is guessing, we MUST NOT have guessing, since it can be or may not be! No bluffing and uncertainty is allowed at any cost! We MUST always know exactly precisely what is happening exactly, in any context, under any conditions, everywhere!"

Tests, gates, status reports, closure narratives, commit messages, and operator-facing text MUST NOT use likely, probably, maybe, might, possibly, presumably, seems, or appears to when describing causes of failures, behaviour, or fix effectiveness. Either prove the cause with captured forensic evidence (`logcat`, `dmesg`, `/sys read-`

ings, getprop, kernel ramoops, dropbox, strace, etc.) and state it as fact, OR explicitly mark UNCONFIRMED: / UNKNOWN: / PENDING_FORENSICS: with a tracked-task ID for follow-up.

Pre-build gate CM-N0-GUESSING-MANDATE greps recently-modified docs

- test scripts for the forbidden vocabulary outside explicit UNCONFIRMED: / UNKNOWN: / PENDING_FORENSICS: blocks. Paired mutation introduces a likely token into a fresh status block → gate FAILs. Propagation gate CM-COVENANT-114-6-PROPAGATION enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.6.

Non-compliance is a release blocker regardless of context.

§11.4.7 — Demotion-evidence rule (Phase 38.X+2 amendment, 2026-05-11)

A demotion from any FAIL classification (OPEN, POSSIBLE PRODUCT DEFECT, FAIL) to a lower-severity classification (INVESTIGATED, MITIGATED, RESOLVED, WORKING-AS-INTENDED) requires positive evidence captured under the **same conditions** that originally exposed the defect — same device, same firmware, same cycle position, same load profile.

"I cannot reproduce in isolation" is a HYPOTHESIS, not a finding. Per §11.4.6 it MUST be tagged UNCONFIRMED: until same-conditions retest produces positive evidence. The expanded forbidden-vocabulary list:

Forbidden phrase	Why it bluffs
"isolated re-run PASSes therefore X was a flake"	Strips the very environment that exposed the defect.
"runtime drift"	Label for "we don't know what changed".
"intermittent" / "transient"	Label for "we don't know how to reproduce".
"pending stress retest"	Defers the actual investigation indefinitely.
"correlates with X"	Hypothesis presented as causation.

Pre-build gate CM-DEMOTION-EVIDENCE-RULE scans Issues.md / Fixed.md / CONTINUATION.md for these phrases outside explicit UNCONFIRMED: / UNATTRIBUTED: / PENDING_CYCLE_RETEST: blocks.

Propagation gate CM-COVENANT-114-7-PROPAGATION enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.7.

Non-compliance is a release blocker regardless of context.

§11.4.8 — Deep-web-research-before-implementation mandate (User mandate, 2026-05-12)

Before designing a non-trivial fix, implementing a new feature, or declaring an architectural choice, perform deep web research to verify the chosen approach is informed by current state-of-the-art. Research surface: official documentation (Android/AOSP/Khronos/CEA-861/AES/IEEE/IETF/ITU), vendor technical guides (Rockchip, Sipeed, Audinate Dante, Synaptics, Realtek, Bluetooth SIG), open-source codebases (Linux kernel, ALSA, Bluez, ExoPlayer, libVLC, MPV, FFmpeg, AOSP forks), coding tutorials + technical articles (Stack Overflow, AOSP Code Lab, AES papers), issue trackers (Android bug tracker, AOSP Gerrit, GitHub issues).

A fix that re-invents a wheel — or reproduces a known-broken pattern — when the open-source community has already solved the problem is a §11.4 violation by omission. Every non-trivial fix's commit / Issues.md / Fixed.md entry **MUST** cite at least one external source URL OR the literal "NO external solution found — original work".

Pre-build gate CM-RESEARCH-CITATION-PRESENT scans new fix-direction blocks for the pattern. Propagation gate CM-COVENANT-114-8-PROPAGATION enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Documentation continuity requirement: every fix landed under §11.4.8 also adds to docs/guides/ a user-facing or developer-facing guide section where appropriate.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.8.

Non-compliance is a release blocker regardless of context.

§11.4.9 — Batch-source-fixes-before-rebuild mandate (User mandate, 2026-05-12)

When closing a multi-defect batch, all source-side fixes that **DO NOT** require runtime on-device validation to design **MUST** be landed **BEFORE** the next firmware rebuild. Anti-pattern elimin-

ated: Fix A → rebuild → flash → cycle → fix B → rebuild → ... serializes 7-8 hours per fix instead of batching all into ONE build cycle. Operator time is the scarce resource.

Exceptions documented in commit message as `REQUIRES_REBUILD:` `<reason>`: kernel-5.10/ changes, atmosphere-*.sh boot-script side-effects, hardware/rockchip/ HAL behavior — each gates downstream state and requires firmware to validate.

Before declaring a batch "ready for rebuild": pre-build GREEN + meta-test GREEN + existing-device validations performed where possible + Issues.md/Fixed.md/CONTINUATION.md in sync (+ HTML/PDF exported) + §11.4.8 research citations all logged.

Propagation gate CM-COVENANT-114-9-PROPAGATION enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.9.

Non-compliance is a release blocker regardless of context.

§11.4.10 — Credentials-handling mandate (User mandate, 2026-05-12)

All credentials, secrets, API tokens, passwords, phone numbers, OAuth tokens, signing keys MUST NEVER live in tracked files. Templates with placeholder values are allowed (.example suffix). Tests load credentials at runtime from scripts/testing/secrets/ (or per-submodule equivalent); operator-populated files are `chmod 600`, directory is `chmod 700`. `.env`, `.env.*`, `*.env` patterns + scripts/testing/secrets/* (with .example + README.md exception) git-ignored project-wide.

Test scripts MUST NEVER echo credentials to stdout/stderr/logcat. Screen- recording of sign-in flows MUST redact credential-bearing frames. Per-service file separation (`.netflix.env`, `.disney.env`, etc.) limits blast radius.

Forensic-rotation policy: suspected leak → rotate at provider, update local `.env`, audit captured artifacts. Pre-build gate CM-CREDENTIAL-LEAK-SCAN greps tracked files for entropy-suspicious password strings + known API-token formats. Propagation gate CM-COVENANT-114-10-PROPAGATION enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.10.

Non-compliance is a release blocker regardless of context.

§11.4.14 — Test playback cleanup mandate (User mandate, 2026-05-13)

Every test that issues `am start / cmd media_session play / MediaController.play` MUST issue matching `am force-stop / input keyevent KEYCODE_MEDIA_STOP` + register cleanup in EXIT trap. Verified via positive evidence (Arvus codec-state → N.E., `dumpsys media_session` shows no PLAYING for test app). `test_all_fixes.sh` post-test sanity check FAILs the just-completed test if it left orphan playback. HelixQA Challenges bound equally. No grace period — "next test will clean it up" is §11.4 PASS-bluff.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.14. Pre-build gates CM-TEST-PLAYBACK-CLEANUP + CM-COVENANT-114-14-PROPAGATION.

Non-compliance is a release blocker regardless of context.

§11.4.15 — Item-status tracking mandate (User mandate, 2026-05-13)

Every active item in `docs/Issues.md` carries a `**Status:**` line with one of six values: Queued, In progress, Ready for testing, In testing, Reopened, Fixed (→ `Fixed.md`). Status MUST be updated as the item progresses through its lifecycle. Fixed requires captured-evidence per §11.4.5 + migration to `Fixed.md`.

The auto-generated `docs/Issues_Summary.md` includes the Status column. All three file types (`.md`, `.html`, `.pdf`) MUST be in sync at all times — enforced by CM-DOCS-EXPORT-SYNC (§11.4.12 + §11.4.15 amendment).

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.15. Pre-build gates CM-ITEM-STATUS-TRACKING + CM-COVENANT-114-15-PROPAGATION.

Non-compliance is a release blocker regardless of context.

§11.4.16 — Item-type tracking mandate (User mandate, 2026-05-14)

Every active item in `docs/Issues.md` carries a `**Type:**` line with one of three values: Bug (product defect / regression / user-visible broken behaviour), Feature (new capability not previously offered to end users), Task (internal workstream — refactor, doc, infra, gate, audit; the lowest-stakes default when ambiguous). The vocabulary is CLOSED — no other value is permitted.

The auto-generated `docs/Issues_Summary.md` includes the Type column. All three file types (`.md`, `.html`, `.pdf`) MUST be in sync at all times — enforced by CM-DOCS-EXPORT-SYNC (§11.4.12 + §11.4.15 + §11.4.16 amendment).

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.16. Pre-build gates CM-ITEM-TYPE-TRACKING + CM-COVENANT-114-16-PROPAGATION.

Non-compliance is a release blocker regardless of context.

§11.4.13 — Out-of-band sink-side captured-evidence mandate (User mandate, 2026-05-13)

Whenever an HDMI sink with a network-accessible introspection API is present (current example: Arvus H2-4D-273 at <http://192.168.4.172/>), the test suite MUST consume the sink's report as captured-evidence for every audio test asserting a codec / channel-count / passthrough mode. On-SoC HAL telemetry ALONE is insufficient — that is the exact "tests pass but the feature doesn't work" pattern §11.4 forbids. Reference: [scripts/testing/lib/arvus_probe.sh](#), [scripts/testing/arvus_probe.sh](#), [docs/guides/ARVUS_HDMI_INTEGRATION.md](#). Pre-build gate CM-ARVUS-EVIDENCE-INTEGRATED (7 invariants) + paired mutation. No hardcoding (env: ARVUS_HOST etc.). Topology dispatch per §11.4.3 — sink unreachable → SKIP, never FAIL. Identity verification (MAC match) before consuming codec-state. Anti-stickiness post-stop. HelixQA Challenges bound equally.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.13. Integration reference: [docs/guides/ARVUS_HDMI_INTEGRATION.md](#).

Non-compliance is a release blocker regardless of context.

§11.4.11 — File-layout discipline (User mandate, 2026-05-12)

Files live in canonical directories per type:

- Shell scripts → [scripts/](#) (legacy: [scripts/legacy/](#))
- Log files → [logs/](#) (legacy: [logs/legacy/](#))
- Release artifacts → [releases/<app>/<version>/](#)
- Operator credentials → [scripts/testing/secrets/](#) (per §11.4.10, git-ignored)
- Markdown docs → [docs/](#) + [docs/guides/](#) + [docs/research/](#) + [docs/superpowers/plans/](#)
- Per-version changelogs → [docs/changelogs/](#)
- Hardware ID photos → [docs/hardware/<device-slug>/](#)

Repo root contains ONLY: AOSP-mandated top-level files (Android.bp, Makefile, bootstrap.bash, BUILD, kokoro, lk_inc.mk, OWNERS, version_defaults.mk), project metadata (README/CLAUDE/AGENTS/CONTRIBUTING/LICENSE/NOTICE/VERSION), dot-files (.gitignore/.gitmodules), and standard top-level

dirs (build/, device/, external/, frameworks/, hardware/, kernel-5.10/, packages/, prebuilts/, scripts/, system/, tools/, vendor/, docs/, releases/, logs/).

NO bash scripts in repo root except AOSP-mandated bootstrap.bash. NO log files in repo root. NO duplicate filenames between root and scripts/. NO release artifacts in root. Moves require triple-verification (audit all references + distinguish absolute vs subdir-local + confirm no AOSP build- system requirement). Pre-build gate CM-FILE-LAYOUT-DISCIPLINE enforces. Propagation gate CM-COVENANT-114-11-PROPAGATION enforces this anchor in every CLAUDE.md / AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.11.

Non-compliance is a release blocker regardless of context.

§11.4.12 — Issues_Summary.md sync mandate (User mandate, 2026-05-12)

docs/Issues_Summary.md is the canonical short-form summary of all open items. MUST be regenerated + re-exported (HTML + PDF) whenever Issues.md changes. Generator: scripts/testing/generate_issues_summary.sh. Pre-build gates CM-ISSUES-SUMMARY-SYNC + CM-COVENANT-114-12-PROPAGATION enforce mechanically.

Sort order (User mandate refinement 2026-05-12): severity DESC (C → M → L), then intra-group criticality DESC inside each group. Most critical row = #1, least critical = #N. Documented at the top of the generated file.

Auto-sync wrapper: scripts/testing/sync_issues_docs.sh — runs generator + export_progress_docs.sh in one shot. MUST be invoked after any edit to Issues.md or Issues_Summary.md. HTML+PDF exports are NEVER manually invoked; they ALWAYS travel with the markdown.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.12.

Non-compliance is a release blocker regardless of context.

§11.4.33 — Type-aware closure-status vocabulary (User mandate, 2026-05-15)

§11.4.15 defined the lifecycle Status closed-set including terminal Fixed (→ Fixed.md). §11.4.16 defined the Type closed-set {Bug | Feature | Task}. §11.4.33 binds the two — closure terminal value MUST agree with the item Type: Bug → Fixed (→ Fixed.md), Feature → Implemented (→ Fixed.md), Task → Completed (→ Fixed.md). The (→ Fixed.md) suffix is preserved so existing migration tooling (atomic

Issues.md → Fixed.md move per §11.4.19) keeps working. Generators treat the three terminal values as semantically equivalent (all closed, positive evidence captured) but preserve the literal in emitted docs. Closing a Feature with Fixed (→ Fixed.md) or a Task with Implemented (→ Fixed.md) is a §11.4.33 violation. Pre-build gate CM-CLOSURE-VOCAB-TYPE-AWARE.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.33.

Non-compliance is a release blocker regardless of context.

§11.4.34 — Reopened-source attribution mandate (User mandate, 2026-05-15)

Every Issues.md heading whose ****Status:**** is Reopened MUST carry a ****Reopened-Details:**** line within 8 non-blank lines of the heading, capturing four sub-facts: **By:** AI or User; **On:** ISO date; **Reason:** one of { test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-re-discovered | design-reconsidered } or explicit free text; **Evidence:** path or short description of the captured artefact. Reopens without evidence are §11.4.6 / §11.4.7 violations: the reopen IS a demotion-from-Fixed change. Issues_Summary.md Status column MUST distinguish Reopened sub-states by source (e.g. Reopened (AI: test-failed) vs Reopened (User: manual-testing)). Pre-build gate CM-ITEM-REOPENED-DETAILS mirrors CM-ITEM-OPERATOR-BLOCKED-DETAILS walk pattern.

Canonical authority: parent [docs/guides/ATMOSPHERE_CONSTITUTION.md](#) §11.4.34.

Non-compliance is a release blocker regardless of context.

§11.4.35 — Canonical-root inheritance clarity (User mandate, 2026-05-15)

The constitution submodule's three files (constitution/Constitution.md, constitution/CLAUDE.md, constitution/AGENTS.md) ARE the canonical root — also called the parent files. Universal rules per §11.4.17 live there.

This submodule's CLAUDE.md and AGENTS.md are consumer extensions of the parent ATMOSphere-Android-15 project, which is itself a consumer extension of the constitution submodule. Both layers open with an inheritance pointer (the **## INHERITED FROM** heading near the top of each, or @constitution/CLAUDE.md for agents that resolve @imports). This submodule's files contain only rules specific to this submodule's role inside ATMOSphere (player wiring, on-device test contract, build-step integration, APK module

name, applicationId conventions). Project-wide ATMOSphere rules live in the parent ATMOSphere-Android-15 CLAUDE.md / AGENTS.md. Universal rules live in the constitution submodule.

When in doubt: universal rule → constitution submodule; ATMOSphere-wide rule → parent project repo root; this-submodule-only rule → this file. Default to the narrowest layer when uncertain. "Parent CLAUDE.md" / "root Constitution" → constitution submodule file at constitution/<filename>, NEVER this submodule's own CLAUDE.md / AGENTS.md and NEVER the parent ATMOSphere-Android-15 project root files. Moving a rule between layers MUST be a visible commit — git mv + an explicit "Lifted from to per §11.4.35" line in the message. AI agents MUST NOT silently re-author a §11.4.X anchor in the wrong layer and call it propagation.

Recommended pre-build gate CM-CANONICAL-ROOT-CLARITY verifies the inheritance pointer is present and the constitution submodule files are reachable. Composes with §11.4.17.

Canonical authority: constitution submodule Constitution.md §11.4.35.

Non-compliance is a release blocker regardless of context.

§11.4.40 — Full-suite retest before release tag mandate (User mandate, 2026-05-17)

A release tag MUST NOT be created until a COMPLETE retest with ALL existing tests has been executed on a clean baseline AFTER every workable item in the batch is done, fixed, polished, and individually verified. Spot-check retests that run only the tests touched by the batch are FORBIDDEN — they miss interaction defects between the batch's fixes and previously-stable code.

The complete retest comprises: (1) pre-build full sweep, (2) post-build full sweep, (3) on-device 4-phase cycle on EVERY owned device, (4) meta-test full mutation sweep, (5) Challenge bank full sweep, (6) Issues.md/Fixed.md state audit, (7) CONTINUATION.md sync check.

Time is essential — complete retest is typically 12-48 hour elapsed effort. NOT optional, NOT abbreviated. Skipping is the exact "tests passed but feature broken" failure mode §11.4 specifically prohibits.

Composes with §11.4.4 (per-fix retest) — §11.4.37 is the additional final integrity check at RELEASE granularity. Composes with §11.4.7 — full-suite retest is the authoritative baseline for closures in the batch. No escape hatch — no --skip-full-retest or --quick-release flag exists.

Pre-build gate CM-FULL-SUITE-RETEST-MANDATE + paired mutation.
Propagation gate CM-COVENANT-114-40-PROPAGATION enforces this anchor in every CLAUDE.md/AGENTS.md across parent + 10 owned submodules + HelixQA dependencies.

Canonical authority: constitution submodule Constitution.md §11.4.37.

Non-compliance is a release blocker regardless of context.

§11.4.41 — Pre-Force-Push Merge-First Mandate (User mandate, 2026-05-17)

Any force-push (`git push --force`, `git push --force-with-lease`, `git push +<ref>`, or equivalent history-rewriting operation on any remote) authorised under §9.2 / CONST-043 MUST be preceded by a mechanical 4-step merge-first pipeline that brings every remote-side commit into the local tree, resolves every conflict carefully, and verifies nothing is lost or corrupted on EITHER side BEFORE the overwriting push is executed.

The 4-step pipeline (mandatory, in order): (1) `git fetch --all --prune --tags` against every configured remote — capture output. (2) Integrate every divergent commit locally via `git rebase` (local is strict superset), `git merge` (independent additions both deserve preservation), or operator-confirmed cherry-pick (remote subset already present locally). (3) Audit: no conflict markers (`grep -rn '^<<<<<< \|^===== $|^>>>>>>'` returns empty), no silent file drops (`git diff --stat HEAD@{1} HEAD`), every previously-passing test still passes per §11.4.4 / §11.4.40 baseline, every captured-evidence artifact still validates. (4) `git push --force-with-lease <remote> <ref>` (NEVER `--force` without `--with-lease` unless §9.2 sub-clause 6 explicitly authorises it for a remote where lease semantics are unavailable). One force-push event per CONST-043 authorisation — no batch authorisation.

Two-gate composition with CONST-043 — §11.4.41 does NOT relax CONST-043's operator-approval requirement. Gate A (CONST-043): operator types explicit per-operation force-push authorisation. Gate B (§11.4.41): agent executes the 4-step merge-first pipeline, captures evidence of clean integration, presents evidence to operator BEFORE the force-push. Both gates required.

Verification artefact — every §11.4.41-governed force-push emits a `docs/changelogs/<tag>.md` "Force-push merge-first audit" section containing 7 elements: (i) `git fetch` output, (ii) per-remote `HEAD..<remote>/<branch>` log before integration, (iii) integration strategy chosen per remote with rationale, (iv) post-integration conflict-marker scan output (must be empty), (v) post-integration

test suite delta (must show only expected changes), (vi) --force-with-lease push output with lease SHA evidence, (vii) CONST-043 authorisation quote from the conversation.

Composes with §9.2 (data-safety hardlinked backup), §11.4.4 (test-interrupt-on-discovery — broken integration triggers rollback), §11.4.6 (no-guessing — every step's outcome captured, not assumed), §11.4.26 (constitution-submodule update pipeline — per-submodule specialisation), §11.4.32 (post-pull validation — audit step's mechanical companion), §11.4.37 (fetch-before-edit — step 1 enforces it for force-push specifically), §11.4.40 (full-suite retest — step 3's test-evidence requirement).

No escape hatch — the operator-pressure escape ("just force-push, we'll fix it later") is the exact failure mode this anchor closes. Pre-build gate CM-COVENANT-114-41-PROPAGATION enforces this anchor in every CLAUDE.md/AGENTS.md across parent + 10 owned submodules + nested submodules + HelixQA dependencies. Paired mutation strips the anchor literal → gate FAILs. Gate CM-FORCE-PUSH-MERGE-FIRST walks docs/changelogs/<tag>.md "Force-push" entries for the 7 audit elements; paired mutation strips any element and asserts gate FAILs.

Canonical authority: constitution submodule Constitution.md §11.4.41.

Non-compliance is a release blocker regardless of context.

§11.4.52 — Autonomous-Validation Mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18):

"Make sure we have full automation tests which will do all this work in full automation! IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well."

Every user-facing feature MUST have at least one autonomous validation path: end-to-end via adb shell + scripted automation, captured runtime evidence per §11.4.5, PASS/FAIL verdict WITHOUT human presence to drive UI, observe screen, or make decisions. Operator-attended tests are SUPPLEMENTARY, never PRIMARY. A feature whose ONLY validation path is operator-attended is a §11.4.52 violation — the path does not scale to CI, does not run on every commit, does not survive operator unavailability, and produces the exact "tests pass but feature doesn't work for users" failure mode §11.4 forbids.

Acceptable autonomous paths: (a) programmatic instrumentation APK (SDK-API exercises like `MediaCodec.createDecoderByName` + structured JSON result file); (b) headless intent dispatch + state poll (`am start -es / am broadcast + dumsys / /proc/<pid>/maps / media.metrics polling`); (c) ADB-driven uiautomator (ONLY if hierarchy has ≥ 1 clickable node — empty hierarchy demands fallback to APK/intent); (d) network-side sink probe per §11.4.13; (e) HelixQA autonomous QA session per §11.4.27.

Coverage ledger (§11.4.25) classifies each feature as `AUTONOMOUS_VERIFIED` / `AUTONOMOUS_DESIGNED` / `OPERATOR_ATTENDED_ONLY` / `NOT_APPLICABLE`. `OPERATOR_ATTENDED_ONLY` blocks release until migrated; cite tracked work item per §11.4.15 + §11.4.16. Autonomous paths themselves MUST be anti-bluff: positive captured evidence + paired meta-test mutation per §1.1.

Composes with §11.4.25 (full-automation coverage), §11.4.27 (no-fakes + 100% type coverage), §11.4.39 (per-feature on-device end-user validation), §11.4.43 (TDD RED-first), §11.4.48 (UI-driven — fallback to APK/intent when uiautomator hierarchy empty), §11.4.49 (dual-approach), §11.4.50 (deterministic consistency), §11.4.51 (live-ADB-first).

Pre-build gates: `CM-COVENANT-114-52-PROPAGATION` + `CM-AF-AUTONOMOUS-PATH-PER-FEATURE`. Paired mutations. No escape hatch — no `--allow-operator-attended-only`, `--skip-autonomous-path`, `--manual-validation-suffices` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.52.

Non-compliance is a release blocker regardless of context.

§11.4.53 — Fixed_Summary parity mandate (User mandate, 2026-05-18)

Forensic anchor — verbatim user mandate

(2026-05-18T17:55Z): "Note: Just like for Issues we have `Issues_Summary`, for Fixed we MUST HAVE `Fixed_Summary` - like all other docs: ALWAYS in sync and up to date and ALWAYS exported into the PDF and HTML! Add this mandatory rule / constraint into the root (constitution Submodule) `Constitution`, `AGENTS.MD` and `CLAUDE.MD`."

`docs/Fixed_Summary.md` is the symmetric short-form summary of `docs/Fixed.md`. MUST be regenerated whenever `Fixed.md` changes. HTML + PDF exports MUST travel with the markdown (identical mtimes within `sync_issues_docs.sh` granularity). Stale exports are §11.4.53 violations regardless of whether the underlying `.md` is correct. Same discipline as §11.4.12 `Issues_Summary` applied to `Fixed.md`.

Generator: `scripts/testing/generate_fixed_summary.sh` (canonical, executable, emits markdown table with Status + Type columns per §11.4.19 column-alignment). Auto-sync wrapper: `scripts/testing/sync_issues_docs.sh` regenerates BOTH summaries in one shot, exports HTML + PDF, colorizes per §11.4.23, re-renders PDFs. MUST be invoked after any edit to `Fixed.md`. No `--issues-only` flag exists, and §11.4.53 prohibits adding one.

Sort order: closure date DESC (most-recent-Fixed first), `$-letter / Fix-#` secondary. Documented at the top of the generated file.

Composes with §11.4.12 (Issues_Summary sibling — canonical pair), §11.4.19 (atomic Issues→Fixed migration trigger + column-alignment), §11.4.23 (colorizer post-processes both summaries), §11.4.33 (type-aware closure vocabulary — Fixed_Summary respects Fixed (→ Fixed.md) / Implemented (→ Fixed.md) / Completed (→ Fixed.md) terminal values), §11.4.44 (revision header applies to Fixed_Summary.md), §12.10 (CONTINUATION.md resumption guarantee).

Pre-build gates: CM-FIXED-SUMMARY-SYNC (6 invariants — Fixed_Summary exists + HTML/PDF mtime ≥ md mtime + Fixed_Summary mtime ≥ Fixed mtime + generator + sync wrapper invokes generator) + CM-COVENANT-114-53-PROPAGATION (anchor literal across canonical files). Paired mutations strip the anchor literal AND move the generator aside AND backdate Fixed_Summary mtime. No escape hatch — no `--skip-fixed-summary-sync`, `--issues-only`, `--summary-not-applicable` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.53.

Non-compliance is a release blocker regardless of context.

§11.4.58 — Parallel-development methodology (User mandate, 2026-05-19)

Project work proceeds through the **Parallel Work Unit (PWU) pipeline** rather than sequential Phase-chain. Each PWU has: ATM-NNN identifier (§11.4.54), Issues.md entry (§11.4.15+§11.4.16), file-scope manifest, §11.4.43 RED test, source patch, pre-build gate, post-flash test, paired §1.1 meta-test mutation, HelixQA Challenge bank entry, captured-evidence directory (§11.4.5+§11.4.52).

5-stage pipeline: Stage 1 DEVELOP (parallel PWU agents in worktrees) → Stage 2 MERGE (serial conductor + §11.4.41 4-step merge-first) → Stage 3 REBUILD+FLASH (parallel where hardware allows) → Stage 4 VALIDATE (parallel D3+D4+meta-test+coverage) → Stage 5 SWEEP (parallel HelixQA + Fixed.md migration + README refresh). Stage 1 of round N+1 overlaps with Stages 4-5 of round N.

Synchronization: 4-layer lock hierarchy (parent flock / per-submodule git / contention-path advisory locks for 10 forbidden cross-PWU paths / per-PWU worktree). Disjoint-scope PWUs fully parallel.

Anti-bluff merge-time enforcement (mandatory, all four): C1 §11.4.43 RED-test captured. C2 §1.1 paired meta-test mutation FAILs the gate. C3 §11.4.50 3-iter (or 10-iter) deterministic-consistency. C4 §11.4.5 captured-evidence per feature type. Metadata-only / configuration-only / absence-of-error / grep-without-runtime PASS REJECTED. HelixQA Challenge bank coverage MANDATORY for every user-visible PWU.

Phase 39.EX infrastructure gates (5 gates land the parallel infrastructure itself): CM-PWU-PARALLEL-VALIDATION-ORCHESTRATOR, CM-PWU-HELIXQA-PER-DOMAIN-RUNNER, CM-PWU-WORKER-POOL-LOCKING, CM-PWU-FILE-SCOPE-PARTITION, CM-PWU-AUTO-MERGE-GATE-6CONDITIONS. Each ships a paired meta-test mutation per §1.1.

Pre-build gates CM-PWU-LOCK-HIERARCHY + CM-PWU-ANTI-BLUFF-COVERAGE

- CM-PWU-MERGE-QUEUE-DISCIPLINE + CM-PWU-PARALLEL-AGENT-LIMIT + CM-COVENANT-114-58-PROPAGATION. Paired mutations cover each gate. No escape hatch.

Canonical authority: constitution submodule [Constitution.md](#)
§11.4.58. Project-specific implementation reference: [docs/guides/PARALLEL_DEVELOPMENT_METHODODOLOGY.md](#).

Non-compliance is a release blocker regardless of context.

§11.4.65 — Universal Markdown export mandate (User mandate, 2026-05-19)

Every Markdown document inside the project that is NOT part of an application or service's source-code tree MUST have synchronized .html and .pdf siblings. Includes: project-root *.md, docs/**/*.md, scripts/**/*.*md (doc-format companion docs), owned-submodule top-level README.md / CLAUDE.md / AGENTS.md / CHANGELOG.md and their docs/**/*.*md, constitution/**/*.*md, owned HelixQA submodules' equivalents. Excludes: external/**, prebuilts/**, packages/modules/**, kernel-5.10/**, out/**, build/**, application/service source-code trees, and third-party submodules NOT in the owned set. Every edit triggers regeneration via scripts/testing/sync_all_markdown_exports.sh (pandoc HTML + weasyprint PDF, timeout 60 per file, capped at 500 candidates). HTML + PDF mtime MUST be $\geq$ source .md mtime at all times.

Pre-build gates CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC + CM-COVENANT-114-65-PROPAGATION. Paired meta-test mutations. Composes with §11.4.12 / §11.4.18 / §11.4.23 / §11.4.44 / §11.4.45 /

§11.4.53 / §11.4.59 / §11.4.60 / §11.4.63 / §11.4.64. No escape hatch — no --skip-md-exports, --no-pdf-only, --md-export-not-applicable flag.

Canonical authority: constitution submodule [Constitution.md](#) §11.4.65.

Non-compliance is a release blocker regardless of context.